Basic Functions of Git & GitHub

Ayan Singh Tomar
Department of Computer Science and Engineering
Manipal Institute of Technology, MAHE
Manipal,India
ayan.mitmpl2026@learner.manipal.edu

Abstract – This task was carried out to understand the version control system Git. The tutorials on the Learn Git Branching website were used to study concepts like commits, branches, merging, rebasing and remote repositories. These concepts were then used on a demo repository containing the solution files of the problems.

Keywords—Git, version control, commits, branching, repositories, merging, rebasing, remote repositories.

I. INTRODUCTION

Software projects undergo numerous changes during development making it very important to record previous versions of the project. Version control systems help us achieve this by providing a structured method for these changes and maintaining project history. Git is a distributed version control system that maintains repositories locally allowing for multiple operations to be performed without the need of being connected to a remote server continuously. Every developer has a full copy of the entire project history on their own machine, not just the latest snapshot. Git provides features such as clone, commit, push, pull, branching, merging and rebasing for managing project history.

II. GIT COMMITS AND HISTORY

A. GIT Commits

A commit represents a saved version of a project at a particular point in time. It records a snapshot of all the tracked files in your directory. Each commit has a reference to a previous commit which enables git to record the history of changes made to a project. When a new commit is created the current branch moves to the new commit and every new commit contains a hardcoded pointer to its parent commit.

B. HEAD and Relative References

HEAD is the name GIT uses to reference the branch that is currently checked out. It essentially indicates what commit you are working on top of, and usually points to the most recent commit. Git also allows previous commits to be referenced using relative references:

- Using HEAD^ and HEAD~1 allows for moving back through commits.
- These relative references are useful as you can then reference previous commits without using the full commit hash (which looks something like: 7af68a4128b936d85a11cda34a9e22303fb1462d).

III. BRANCHING AND BRANCH MANAGEMENT

A. What a Branch really is

A branch like main or feature is not a separate copy of the whole repository. It is simply a lightweight, movable pointer to a specific commit. It is just a small file inside .git that stores the hash code of one commit.

B. Essential Branch Commands

- git branch: Shows all local branches in your repository and highlights your active branch with an asterisk.
- git branch <branch-name>: Creates a new pointer at your current commit but does not switch your active workspace to it.
- git switch <branch-name>: Moves your active workspace (the HEAD pointer) to the specified branch.
- git switch -c <branch-name>: A shortcut that creates a brand-new branch and switches you to it immediately.

IV. MERGING AND REBASING

A. Merging

Merging is the feature on Git that enables us to bring changes from one branch onto another branch. Git handles this in two primary ways:

- If the branch being merged into has not moved since the other branch was created, Git can do what is called a fast-forward merge, where it just moves the pointer forward and no new commit is created.
- If both branches have new commits of their own, Git does a three-way merge instead where it looks at the last common commit of both branches, compares the changes made on each side, and creates one new commit that has two parents

B. Rebasing

Rebasing is a process of moving or combining a sequence of commits to a new base commit. For instance, running the command git rebase main will cause every commit that is unique for the current branch to be reapplied on top of the latest commit of the main branch. Rebasing creates a linear history, which can make the commit history easier to read and understand. As rebasing changes commit hashes, this technique must not be applied to the branch that was already pushed and is being used by other people.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

V. REMOTE REPOSITORIES: PUSH & PULL

A remote repository is a version of your project that is hosted on the internet, a network, or an external server rather than on your local computer.

A. Fetch vs Pull

- The git fetch command is used to fetch all changes from the remote repository to the local repository. It does not touch the files in the working directory, hence why it is also safe to run.
- The git pull command is used to fetch all changes from the remote repository to the current working directory. It is basically a fetch followed by a merge of that updated remote branch into the current branch

B. Force Pushing

A normal git push only works if it just adds new commits on top of what the remote already has. git push --force (or -f) bypasses this safety check, completely overwriting the remote repository's history with whatever is on your local machine.

VI. ADVANCED HISTORY TRACKING AND UNDO OPERATIONS

A. git log and Commit Ancestry

The git log command follows the commit graph starting from the HEAD pointer and traversing backwards through the parent commits to display the permanent history of the branch. Using flags such as --oneline, --graph, and --all provides a clear visual representation of the branch topology and merge structure directly within the terminal.

B. git reflog

While git log shows the chronological ancestry of the current commit, git reflog acts as a local chronological ledger of every location the HEAD pointer has occupied. It includes all checkouts, commits, merges, rebases, and resets in the order they were executed. Because commands like reset and rebase can detach and leave old commits behind, git reflog entries are not strictly connected, making it an invaluable tool for recovering seemingly lost commits. Reflog entries exist exclusively on the local machine and are never pushed to a remote server.

C. Git Revert, Restore, and Tagging

Beyond checking history, Git provides specific mechanisms for safely undoing modifications and marking milestones:

- **Git Revert:** The git revert command undoes changes introduced by a specific previous commit by generating a completely new commit that applies the exact inverse changes. Unlike destructive commands like git reset, git revert does not alter or erase existing commit history, making it the safest option for rectifying mistakes on public or shared remote branches.
- **Git Restore:** The git restore command acts as a modern utility to discard local edits. Executing git restore <file> discards working directory edits entirely, while git restore --staged <file> removes a file from the staging area while preserving the local edits.
- **Git Tag:** Tags permanently mark specific commits as crucial milestones. Unlike branches, tags are anchored to a specific spot in the commit tree and never move as new commits are appended.

VII. CONFLICT RESOLUTION AND STASHING MECHANISMS

A. Resolving Merge Conflicts

A merge conflict occurs when Git attempts to combine two branches that have modified the identical line of a file, or when one branch edits a file that the other branch deletes, preventing automatic resolution. When a command like git merge is executed under these conditions, Git pauses the merge and marks the conflicting file segments with specific text delimiters.

The file will display <<<<<<< HEAD to denote the beginning of the current branch's version, followed by ======= to separate the conflicting changes, and >>>>>>> <branch-name> to mark the incoming branch's version. Resolving the conflict requires the developer to manually edit the file, retain the correct code logic, and delete all three Git marker lines. Once the file reflects the intended final state, executing git add <filename> marks the conflict as resolved, and git commit finalizes the operation by generating the merge commit.

B. Git Stashing

The git stash command temporarily shelves uncommitted changes in the working directory. This feature is vital when a developer needs to abruptly switch branches (e.g., to address a critical bug) or pull upstream updates, but is not ready to commit incomplete code.

- git stash list displays all saved stashes.
- git stash pop retrieves and applies the most recent stash while simultaneously removing it from the stash list.
- git stash apply retrieves the changes but retains a backup of the stash for potential reuse.

VIII. PRACTICAL IMPLEMENTATION AND DEMONSTRATION

A. Foundation and Repository Initialization

The practical implementation commenced by utilizing the Learn Git Branching platform to establish a conceptual understanding of branching, rebasing, and remote operations. Subsequently, a local repository was initialized (git init) within a directory containing Python solution files.

These files were staged via git add . and permanently recorded into the repository history using git commit. This local repository was successfully linked to a remote GitHub repository using git remote add origin, and the initial commit was uploaded via git push -u origin main.

B. Branch Divergence and Fast-Forward Integration

To demonstrate isolated development, a new secondary branch was created and checked out. Code modifications (such as updating dictionary sorting logic) were committed exclusively to this branch. Upon switching back to the primary branch, git merge was executed. Because the primary branch had not received any new commits since the secondary branch was created, Git executed a fast-forward merge simply moving the main branch pointer forward without generating an explicit merge commit.

C. Simulating and Resolving Merge Conflicts

To simulate a real-world integration conflict, a new branch (e.g., conflict-branch) was established, and a specific line in a Python script was edited and committed. The environment was switched back to the primary branch, where the exact same line in the identical file was altered with a different string and committed.

Initiating a merge between these two diverged branches predictably triggered a merge conflict. The conflicting Python script was opened manually to locate the <<<<<<< HEAD and >>>>>>> markers. The competing lines of code were evaluated, the desired logic was retained, and the conflict markers were deleted. The resolution was finalized by staging the corrected file and executing a merge commit.

D. Remote Synchronization via Fetch and Pull

To test remote fetching protocols, changes were applied directly to the GitHub remote repository (such as adding CSV dataset files or test comments) to simulate another developer pushing code. Executing git pull origin main successfully fetched these external commits and automatically performed a fast-forward merge into the local repository, proving that local and remote histories could be seamlessly synchronized.

IX. CONCLUSION

The execution of this study provided a profound, hands-on comprehension of Distributed Version Control Systems. Moving beyond mere command syntax, the exercise highlighted Git as a robust architecture built upon commits, lightweight branch pointers, and secure history trees. Interactive visualization of complex operations, particularly rebasing and branching topologies, clarified concepts that are traditionally difficult to interpret from static documentation alone.

Furthermore, encountering and resolving practical errors such as initializing repositories in incorrect directories or managing rejected pushes due to desynchronized local and remote histories served to reinforce the strict protocols Git enforces. Deliberately inducing and resolving a merge conflict demonstrated exactly how concurrent engineering teams safely negotiate overlapping codebase modifications. Ultimately, this implementation confirmed that mastering Git is critical for maintaining organized, parallel, and scalable software development workflows

X. ACKNOWLEDGMENT

Acknowledgment is extended to the Driverless Subsystem seniors and technical leads for providing the necessary guidance, resources, and repository frameworks that facilitated a comprehensive understanding of the Git version control system and the operational concepts detailed in this report.

XI. REFERENCES

[1] Learn Git Branching. [Online]. Available: https://learngitbranching.js.org/

[2] Git, "Git Reference" [Online]. Available: https://git-scm.com/docs